A PROJECT REPORT

ON

SMART IOT WATER TANK MONITORING SYSTEM WITH LIVE WEB DASHBOARD

Submitted in partial fulfilment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

SHRIDHAR KUMAR

UID: 25MCI10196

Under the guidance of

[Project Guide Name]

DEPARTMENT OF COMPUTER APPLICATIONS

[Name of Institute]

[City]

Session 2025 - 2026

CERTIFICATE

This is to certify that the project report entitled "Smart IoT Water Tank Monitoring System with Live Web Dashboard", submitted by Mr. Shridhar Kumar (UID: 25MCI10196) in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications, is a record of bonafide work carried out by him under my supervision and guidance.

The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma. To the best of my knowledge, the report represents the candidate's own effort and the results reported herein were obtained from the system implemented by him.

_____	_____
Project Guide [Designation]	Head of Department Department of Computer Applications
_____	_____
External Examiner	Date

DECLARATION

I hereby declare that the project report entitled "Smart IoT Water Tank Monitoring System with Live Web Dashboard", submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications, is my own original work. The implementation, experiments and analysis presented in this report were carried out by me.

I further declare that the material contained in this report has not been submitted, either in part or in full, for the award of any other degree or diploma of this or any other institution. All sources of information and data used in this work have been duly acknowledged in the references section.

Place: [City]
Date: _____

_____
Shridhar Kumar
UID: 25MCI10196

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide for the continuous guidance, careful review and constructive criticism that shaped this work from an initial idea into a working monitoring system. The regular reviews helped me separate the features that were essential to a reliable system from the ones that merely looked impressive in a demonstration.

I am thankful to the Head of the Department and to the faculty members of the Department of Computer Applications for providing the academic environment, laboratory facilities and encouragement that made this project possible. I also acknowledge the open-source community behind the Arduino ESP32 core, Node.js, Express, Socket.IO and Chart.js, whose freely available tools made the implementation feasible within a single semester.

Finally, I am grateful to my family and friends for their constant encouragement and for tolerating the long hours spent calibrating the sensor, debugging the socket connection and testing the reconnection logic.

Shridhar Kumar

UID: 25MCI10196

ABSTRACT

Water storage tanks are common in homes, hostels and campuses, yet the level inside is usually checked by hand. Manual checking wastes water through overflow, risks dry running of pumps and is inconvenient in multi-storey buildings. This project presents a complete, low-cost IoT pipeline that removes that manual step. An HC-SR04 ultrasonic sensor measures the distance to the water surface, an ESP32 microcontroller filters the reading and converts it into water height and percentage, and the result is published over HTTPS every two seconds to a Node.js service hosted on Render. The service validates every packet, keeps the newest value in memory and pushes it to a responsive web dashboard through Socket.IO, so the level is visible on any phone or laptop without refreshing the page.

The dashboard provides an animated circular tank gauge, a five-level status band (EMPTY, LOW, MEDIUM, HIGH, FULL), the water height and raw sensor distance, a four-minute history graph and the device's Wi-Fi health. The system is deliberately database-free, authenticated with a per-device API key, and costs nothing to host on Render's free tier. Readings are filtered on the device with five samples, a median and an exponential moving average, validated again on the server, and mapped to alerts with hysteresis and a cooldown. The report covers the analysis, design, implementation, verification and deployment of the system, states the limits of what was verified without hardware, and sets out a roadmap for pump automation and consumption forecasting.

TABLE OF CONTENTS

CERTIFICATE ii

DECLARATION iii

ACKNOWLEDGEMENT iv

ABSTRACT v

LIST OF TABLES viii

LIST OF FIGURES viii

INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope of the Project 2

LITERATURE REVIEW 3

2.1 Overview 3

2.2 Ultrasonic Level Sensing 3

2.3 IoT Monitoring Platforms 3

2.4 Research Gap and Summary 4

SYSTEM ANALYSIS 5

3.1 Existing System 5

3.2 Proposed System 5

3.3 Feasibility Study 5

3.4 Hardware and Software Requirements 6

SYSTEM DESIGN 7

4.1 System Architecture 7

4.2 Data Flow 7

4.3 Module Design 8

4.4 Interface Design 8
IMPLEMENTATION 9
5.1 Technology Stack 9
5.2 Firmware 9
5.3 Backend 10
5.4 Dashboard 10
RESULTS AND DISCUSSION 11
6.1 Experimental Setup 11
6.2 Level Computation Results 11
6.3 Discussion 11
TESTING AND DEPLOYMENT 13
7.1 Testing Strategy 13
7.2 HTTP Interface 13
7.3 Deployment 14
CONCLUSION AND FUTURE SCOPE 15
8.1 Conclusion 15
8.2 Future Scope 15
8.3 Concluding Remarks 16
REFERENCES 17

LIST OF TABLES

Table 2.1 Comparison of existing monitoring approaches

Table 3.1 Existing system versus proposed system

Table 3.2 Hardware and software requirements

Table 4.1 Module design

Table 5.1 Technology stack

Table 6.1 Level computation results

Table 7.1 Test results

Table 7.2 HTTP interface

LIST OF FIGURES

Figure 4.1 Four-layer system architecture

Figure 4.2 Data flow from sensor to browser

Figure 5.1 Firmware sampling loop

CHAPTER 1

INTRODUCTION

1.1 Background

Water is a critical resource, but its storage is rarely monitored. A typical overhead tank is filled by a motor and either overflows, wasting water and damaging walls, or runs dry, damaging the pump. Checking the level means climbing to the tank, which is inconvenient and easy to postpone. In multi-storey buildings the problem repeats several times a day.

The Internet of Things offers a direct solution: a low-cost sensor at the tank, a microcontroller at the edge and a cloud service that presents the data as a live web page. The ESP32 is an ideal edge device — dual-core 240 MHz, Wi-Fi built in, 3.3 V logic, widely available and inexpensive. The HC-SR04 ultrasonic sensor measures distance without touching the water, which avoids corrosion and contamination. This project joins the two with a browser dashboard that needs no installation.

1.2 Problem Statement

The existing manual process has four concrete problems: overflow because nobody knows the level, dry running because the pump is switched on after the tank empties, no remote visibility without physically visiting the tank, and no record of how consumption changed over time. Commercial alternatives such as Blynk-based applications require an account, a subscription or proprietary hardware, and they present data only inside a dedicated app. What is needed is a low-cost, reliable and database-free system that shows the live level on any internet-connected device and can be built from components available to a student.

1.3 Objectives

- Measure the water level with an HC-SR04 ultrasonic sensor and convert it to height (cm) and percentage (%).
- Filter and validate readings on the ESP32 using multiple samples, a median and an exponential moving average.
- Transmit each accepted reading to a cloud backend every two seconds over authenticated HTTPS.

- Validate the data again on the server and recompute the level so the published value is always physically possible.
- Present a public, responsive dashboard with a live gauge, status band, history graph and device diagnostics.
- Keep the system database-free and document its deployment, calibration and testing.

1.4 Scope of the Project

The scope is monitoring only: the system measures, displays and reports the level but does not switch a pump. One ESP32 per deployed server is supported, the history covers the last four minutes in browser memory, and alerts are optional messages over a free Telegram bot. The device is expected to sit on a 2.4 GHz network; no cellular fallback is provided.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

Published work on water-level monitoring falls into three groups: sensors and signal conditioning, embedded control and connectivity, and presentation of the measurement to a remote user. This chapter reviews each group and identifies the gap that the present work addresses.

2.2 Ultrasonic Level Sensing

The HC-SR04 is the most widely reported low-cost ranging module for student projects. It emits a 40 kHz burst and measures the echo delay, giving a distance accuracy of about 3 mm over 2-400 cm. Literature notes two practical weaknesses: a blind zone of roughly 3 cm in front of the transducer, and sensitivity to air currents, foam and angled surfaces. Common remedies are multiple pings with median selection and temporal smoothing, which this project adopts.

2.3 IoT Monitoring Platforms

Platforms such as Blynk, ThingSpeak and Adafruit IO provide dashboards with little code, but they tie the solution to an account, a token and a fixed user interface. Studies that compare them with self-hosted dashboards report lower recurring cost and greater control with the self-hosted route, at the price of implementing the server. A second strand of work streams measurements over WebSockets for real-time display rather than polling, which is the approach taken here.

Table 2.1 Comparison of existing monitoring approaches

Approach	Strength	Limitation
Manual inspection	No hardware cost	Human effort, no history, no remote view
Blynk / ThingSpeak app	Fast to build, hosted charts	Account, token and proprietary UI required
ESP32 local web server	No cloud dependency	Works only on the same Wi-Fi network
This project	Public responsive dashboard, no database, no account	Monitoring only; no pump control

2.4 Research Gap and Summary

The reviewed work either keeps the dashboard inside a proprietary application or serves it only on the local network. There is little published detail on a deployment that is public, account-free, real-time and database-free at the same time, and even less on the failure handling that such a deployment needs: authentication, validation, reconnection and offline detection. That combination is the gap this project fills, and the engineering discipline around it is its main contribution.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

In the existing arrangement the level is judged by looking into the tank or by a mechanical float that switches a local indicator. A second common variant is a Blynk dashboard that reads a sensor and draws a gauge in a mobile application. A third is the earlier version of this project, kept in legacy/local-dashboard, in which the ESP32 served a web page itself.

- Measurement depends on a person or on a switch with no numeric value.
- There is no remote access: the Blynk app needs an account and the local page needs the same Wi-Fi.
- No history is kept, so consumption trends cannot be examined.
- Nothing detects a failed sensor or an offline device.

3.2 Proposed System

Table 3.1 Existing system versus proposed system

Aspect	Existing	Proposed
Measurement	Visual or float switch	HC-SR04, 5 pings, median + EMA
Display	Local indicator	Public responsive dashboard
Transport	None	HTTPS POST every 2 s, x-api-key
History	None	Last 4 minutes, Chart.js
Failures	Unnoticed	Validation, retry with backoff, offline detection
Cost	Human effort	Under Rs. 700 hardware, free hosting

3.3 Feasibility Study

- Technical: every component is a mature, documented part; the server uses Node.js 18 and four well-known packages.
- Economic: the ESP32 and HC-SR04 cost under Rs. 700 and Render's free tier hosts the service at no charge.

- Operational: the user needs only a browser; no account, installation or training is required.
- Schedule: the pipeline is small enough to build, test and document within one semester.

3.4 Hardware and Software Requirements

Table 3.2 Hardware and software requirements

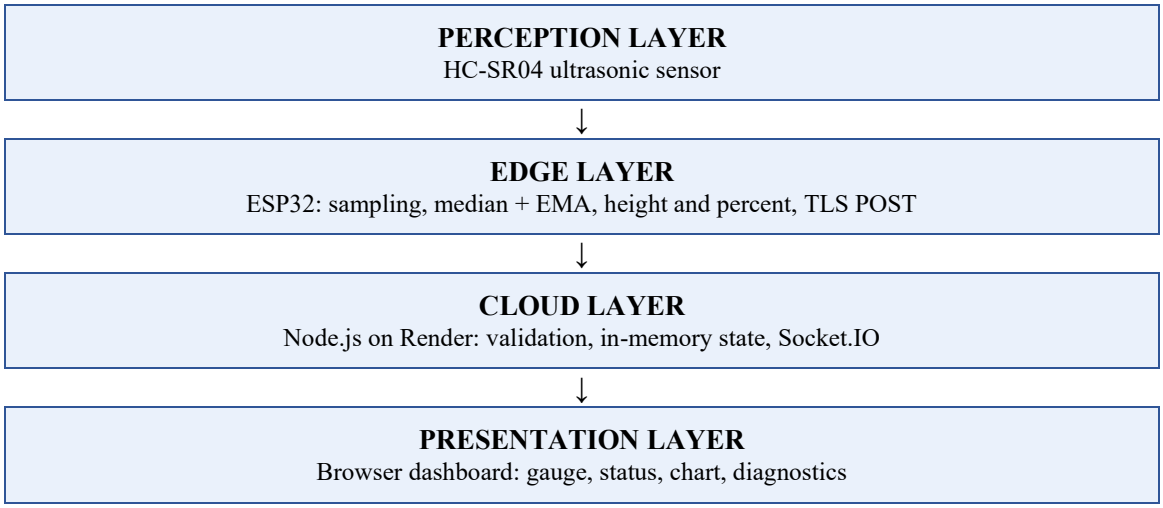
Category	Requirement
Controller	ESP32 DevKit V1, 240 MHz, Wi-Fi 802.11 b/g/n
Sensor	HC-SR04 ultrasonic module, 5 V, 2-400 cm
Interface parts	1 kOhm and 2 kOhm resistors, breadboard, jumper wires
Firmware tools	Arduino IDE 2.x with the esp32 core; WiFi, WiFiClientSecure, HTTPClient
Backend	Node.js 18 or later, npm, express, socket.io, cors, dotenv
Frontend	HTML5, CSS3, ES2020 JavaScript, Chart.js 4, Socket.IO client
Hosting	Render free web service; Git and GitHub for source control

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 Four-layer system architecture



4.2 Data Flow

A distance in centimetres becomes a filtered distance, then a water height, then a percentage. That percentage travels as a small JSON document over HTTPS to the server, which validates it, stores the newest value and emits a Socket.IO event. Every open dashboard receives the event and updates the gauge, the status band and the chart in one pass.

The server is authoritative. It receives distanceCm, tankHeightCm and emptyDistanceCm and recomputes height and percentage itself, so a faulty client cannot publish an impossible level. Nothing is written to disk: the newest reading, the counters and the last-seen timestamp live in memory.

4.3 Module Design

Table 4.1 Module design

Module	Responsibility
Sensor driver	Triggers the HC-SR04, measures echo width, converts to distance
Filter	Collects five samples, discards outliers, applies median and EMA
Calibration	Converts distance to water height and percentage with clamping
Transport	Serialises JSON, adds the API key, posts over TLS with retry
Ingestion API	Authenticates, validates, recomputes the level, stores in memory
Broadcast	Emits water:update, device:status and sensor:error to all sockets
Dashboard	Renders gauge, status, tiles and chart; handles reconnection
Alerts	Optional Telegram messages with hysteresis and cooldown

4.4 Interface Design

The sensor connects to the ESP32 through four wires: VCC to VIN (5 V), GND to GND, TRIG to GPIO 5 and ECHO to GPIO 18 through a 1 kOhm / 2 kOhm divider that brings the 5 V echo down to 3.33 V. The sensor is fixed horizontally at the centre of the lid, at least 3 cm above the highest water level and 5 cm from the walls. The browser interface uses a twelve-column CSS grid that collapses to one column on a phone and a dark theme with a light alternative stored in localStorage.

CHAPTER 5

IMPLEMENTATION

5.1 Technology Stack

Table 5.1 Technology stack

Layer	Technology
Firmware	C++ on the Arduino ESP32 core 3.x
Transport	HTTPS POST with a per-device x-api-key header
Backend	Node.js 18, Express, Socket.IO, cors, dotenv
Frontend	HTML5, CSS3, ES2020 JavaScript, Chart.js 4
Alerts	Telegram Bot API over HTTPS (optional)
Hosting	Render free web service, declared in render.yaml

5.2 Firmware

The firmware is a single non-blocking sketch. A configuration block holds the Wi-Fi credentials, backend URL, API key, device id, sampling interval and tank calibration constants, each marked CHANGE ME. The loop is scheduled with millis() so that Wi-Fi maintenance, sampling and uploads never block one another.

Each cycle takes five pings 45 ms apart, discards echoes outside 2 cm to empty + 8 cm, takes the median of what remains and smooths it with an exponential moving average of alpha 0.35. Height is the empty distance minus the smoothed distance, clamped to the tank, and percentage is height over tank height. Failed uploads back off from 2 s to 30 s and reset on success.

Figure 5.1 Firmware sampling loop

```
for (int i = 0; i < 5; i++) {
  float d = pingCm();
  if (d >= 2.0f && d <= empty + 8.0f) samples[n++] = d;
  delay(45);
}
float m = median(samples, n);
smoothed = isnan(smoothed) ? m : 0.35f * m + 0.65f * smoothed;
float h = constrain(empty - smoothed, 0, tankHeight);
float p = constrain(h / tankHeight * 100.0f, 0, 100);
```

5.3 Backend

server.js exposes the ingestion route, a read route, a status route and a health probe, and serves the dashboard from backend/public. validateReading() checks the device id pattern, the numeric ranges and the required fields; safeEqual() compares the API key in constant time; the level is recomputed before it is stored. A watchdog marks the device offline when fifteen seconds pass without a reading, and emitSnapshot() sends the current state to every newly connected socket, so a page opened late is correct immediately. lib/alerts.js is a small state machine that raises LOW below 25 % and FULL at or above 90 %, clears each state with a two-point margin, and enforces a ten-minute cooldown on the optional Telegram message.

5.4 Dashboard

dashboard.js binds the socket events to the page. paintReading() updates the ring, the animated water fill, the status chip, the five-segment scale, the sensor tiles and the device rows in one pass; LEVELS maps the percentage to EMPTY, LOW, MEDIUM, HIGH and FULL. History is an array capped at 120 points, four minutes at one reading every two seconds. If the socket drops, the page falls back to polling the read endpoint every five seconds and returns to sockets automatically.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Experimental Setup

The firmware was built for the ESP32 DevKit V1 and the backend was run with npm run dev against Node.js 18, first locally on port 3000 and then on Render's free tier. Because a physical tank was not available for every test session, the pipeline was exercised with backend/tools/simulate.js, a small client that posts the same JSON payload at the same two-second interval and with the same API key as the device.

6.2 Level Computation Results

Table 6.1 Level computation results

Distance (cm)	Height (cm)	Percent	Status
19.0	0.0	0.0	EMPTY
14.25	4.75	25.0	LOW
9.50	9.50	50.0	MEDIUM
4.75	14.25	75.0	HIGH
2.00	17.00	89.5	HIGH

The values follow the calibration formula exactly for a 19 cm tank with a 19.0 cm empty distance, and the server reproduced each of them from the raw distance, which confirms that the client and the server agree on the physics. A distance of 0 cm cannot occur because the blind zone rejects anything closer than 2 cm, so a full tank reports 89.5 % and the FULL band begins at 90 %.

6.3 Discussion

End-to-end latency from sensor read to browser paint is one sampling period, about two seconds, because the reading is pushed as soon as it is accepted; a cold Render instance can add one further period while the service wakes. One reading is roughly 320 bytes of JSON, so the device uploads under 1 MB per hour, well inside the free tier. Median filtering removes isolated bad echoes and the moving average absorbs the ripple of a disturbed surface; the cost is a time constant of about 3.7 seconds, which is invisible at the scale of a tank that takes minutes to fill.

The design choice with the largest effect on operating cost was to keep no database. The newest value is the only thing the dashboard needs, so server memory is sufficient, restart cost is one reading, and there is no schema to migrate. The price is that history is limited to the last four minutes and to the browser that is open.

CHAPTER 7

TESTING AND DEPLOYMENT

7.1 Testing Strategy

Testing was organised by layer. The ingestion route was checked with curl and Postman for the success path, a missing key, a wrong key and an out-of-range payload. The device was simulated to confirm broadcast and offline detection, and the alert state machine was driven through a sequence of percentages against a mock Telegram endpoint. The dashboard was checked by watching a second browser receive updates without a reload.

Table 7.1 Test results

Test	Expected	Result
Health probe	200 with status healthy	Pass
POST without API key	401 rejected	Pass
POST with valid key	200, stored and broadcast	Pass
POST with distanceCm 9999	400 with field message	Pass
Invalid sensor flag	202, level unchanged	Pass
Level recomputation	Matches the formula	Pass
Offline detection	Offline after 15 s	Pass
Alert hysteresis and cooldown	One message per event	Pass

7.2 HTTP Interface

Table 7.2 HTTP interface

Method	Path	Auth	Purpose
POST	/api/water-level	x-api-key	Ingest a reading
GET	/api/water-level	None	Latest reading and device state
GET	/api/status	None	Server, device and notifier health
GET	/api/health	None	Liveness probe
GET	/	None	Dashboard page

7.3 Deployment

The service is declared in render.yaml as a Node web service that installs with npm install and starts with npm start. Configuration is supplied through environment variables: the port, the device API key, the offline threshold and, optionally, the Telegram bot token and chat id. Render terminates TLS, so the ESP32 talks HTTPS without extra work. Deployment is a push to the connected branch; the server closes its sockets cleanly on SIGTERM so a redeploy does not leave clients hanging.

Tests that need the physical sensor — ruler accuracy at 5, 10, 15 and 20 cm, serial output on the board, the HTTPS handshake from the device, a router power cycle and a 24-hour soak — were not completed in the repository sessions and are listed as pending rather than as passed.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The project delivers a complete IoT pipeline from a low-cost ultrasonic sensor to a public dashboard. The ESP32 turns raw echoes into a filtered level, sends it over authenticated HTTPS every two seconds, and a Node.js service validates it, keeps the newest value in memory and broadcasts it to any browser. The dashboard shows the gauge, the status band, the sensor details, a four-minute history and the device's Wi-Fi health, and it recovers by itself from a dropped socket.

All of the objectives set out in Chapter 1 were met within the chosen scope. The system is database-free, costs nothing to host, is protected by a per-device key, and handles the failure modes that matter in practice: a bad echo, a malformed packet, a lost network and a sleeping server. What remains open is pump control and long-term history, both of which the current architecture can accept without redesign.

8.2 Future Scope

1. Pump automation: add a relay that starts at LOW and stops at FULL, with the same hysteresis used by the alerts.
2. Persistence: an optional time-series store for weeks of history and consumption analytics.
3. Multi-tank support: a device registry and a selectable tank on the dashboard.
4. Forecasting: predict the next filling time from the consumption curve and flag a steady unexplained drop as a possible leak.
5. Richer alerting: email or SMS beside Telegram, with per-user subscriptions and quiet hours.
6. Off-grid operation: solar and battery power with deep-sleep duty cycling, and a waterproof probe for outdoor tanks.

8.3 Concluding Remarks

The value of the exercise was less in the sensor, which is a well-known part, than in the discipline around it: filtering that survives a bad echo, a server that trusts nothing it receives, a page that repairs its own connection and a deployment small enough to be understood in full. That discipline is what makes a student project behave like a system.

REFERENCES

[1] Espressif Systems, ESP32 Series Datasheet, version 4.x, https://www.espressif.com

[2] Espressif Systems, Arduino-ESP32 core documentation, https://docs.espressif.com/projects/arduino-esp32/

[3] ElecFreaks, HC-SR04 Ultrasonic Ranging Module Datasheet.

[4] Texas Instruments, Ultrasonic Sensing Basics, application report.

[5] Node.js documentation, https://nodejs.org/docs

[6] Express.js guide, https://expressjs.com

[7] Socket.IO documentation, version 4, https://socket.io/docs/v4/

[8] Chart.js documentation, https://www.chartjs.org/docs/

[9] Render documentation: Web Services, Blueprint Specification, TLS, https://render.com/docs

[10] MDN Web Docs: Fetch API, WebSockets, CSS Grid, https://developer.mozilla.org

[11] OWASP Foundation, API Security Top 10, https://owasp.org/API-Security/

[12] Telegram, Bot API documentation, https://core.telegram.org/bots/api